LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 05

SINGLY LINKED LIST (II)



Disusun oleh:

MANGGALA PATRA RADITYA

109082500179

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL

MODUL 05

Link Repo Github Modul 5 :
https://github.com/manggala-patra/Praktikum_StrukturData_Manggala-Patra-Raditya

1. **Soal 1**

   Source Code (SingleLinkedList.h)

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk membuat node
baru dan mengembalikan alamat node tersebut
void dealokasi(ElmList* &P);
void printInfo(List L);

// Searching
ElmList* findElm(List L, int nilai);

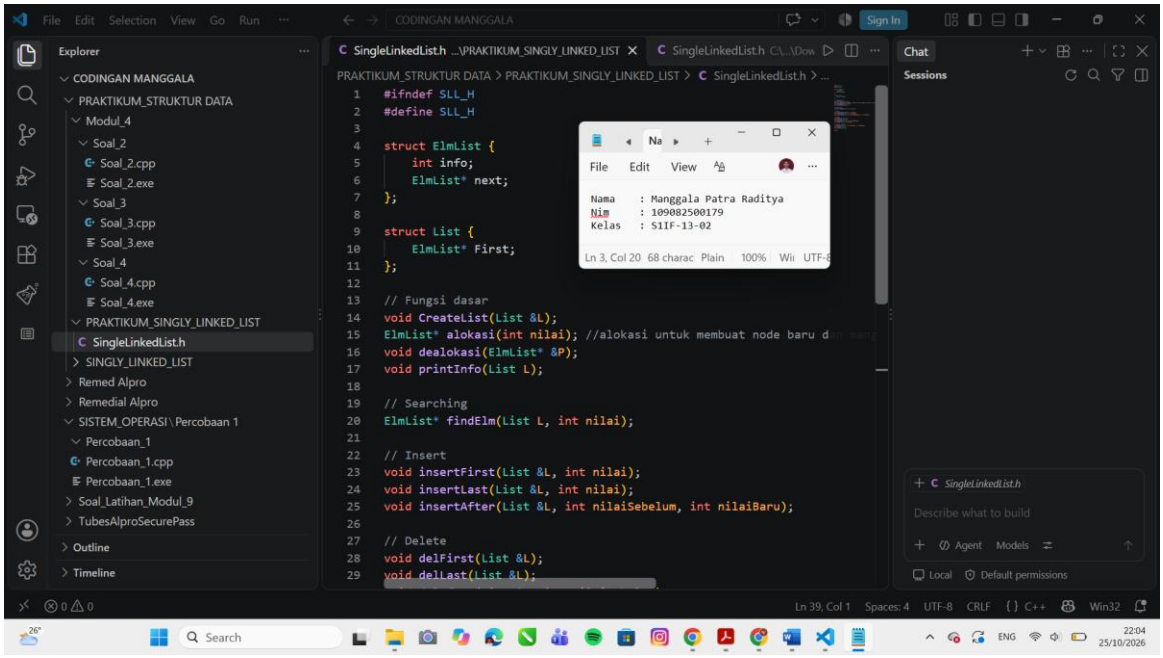
// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

// Delete
void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

// Operasi tambahan
void update(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);
```

```
#endif
```

Screenshoot program



Deskripsi program

File Singlylist.h merupakan file header yang berfungsi untuk mendefinisikan struktur data dan mendeklarasikan fungsi-fungsi yang digunakan dalam program Singly Linked List. Struktur ElmList memiliki anggota info untuk menyimpan data bertipe integer dan next sebagai pointer menuju node berikutnya, sedangkan struktur List memiliki pointer First yang menunjuk ke node pertama. File ini juga mendeklarasikan berbagai fungsi, yaitu CreateList untuk membuat list kosong, alokasi untuk membuat node baru, dealokasi untuk membebaskan memori, printInfo untuk menampilkan data, findElm untuk mencari elemen, insertFirst, insertLast, dan insertAfter untuk menambahkan elemen, delFirst, delLast, delAfter, dan delP untuk menghapus elemen, update untuk mengubah nilai data, nbList untuk menghitung jumlah node, serta delAll untuk menghapus seluruh elemen dalam list. Deklarasi tersebut menjadi dasar bagi implementasi fungsi-fungsi pada file .cpp.

Source Code (SingleLinkedList.cpp)

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}
```

```
// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;

    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    P->next = L.First;
```

```
    L.First = P;
}

// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    if (L.First == nullptr) {
        L.First = P;
    } else {
        ElmList* Q = L.First;

        while (Q->next != nullptr) {
            Q = Q->next;
        }

        Q->next = P;
    }
}

// Menambahkan node setelah nilai tertentu
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
    ElmList* Q = findElm(L, nilaiSebelum);

    if (Q == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else {
        ElmList* P = alokasi(nilaiBaru);

        P->next = Q->next;
        Q->next = P;
    }
}

// Menghapus node pertama
void delFirst(List &L) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else {
        ElmList* P = L.First;

        L.First = L.First->next;
        dealokasi(P);
    }
}

// Menghapus node terakhir
void delLast(List &L) {
```

```
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else if (L.First->next == nullptr) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next->next != nullptr) {
            P = P->next;
        }

        ElmList* Q = P->next;
        P->next = nullptr;
        dealokasi(Q);
    }
}

// Menghapus node setelah nilai tertentu
void delAfter(List &L, int nilaiSebelum) {
    ElmList* P = findElm(L, nilaiSebelum);

    if (P == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else if (P->next == nullptr) {
        cout << "Tidak ada node setelah nilai tersebut." <<
endl;
    } else {
        ElmList* Q = P->next;

        P->next = Q->next;
        dealokasi(Q);
    }
}

// Menghapus node berdasarkan nilai
void delP(List &L, int nilai) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else if (L.First->info == nilai) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next != nullptr &&
               P->next->info != nilai) {
            P = P->next;
        }
```

```
            if (P->next == nullptr) {
                cout << "Nilai tidak ditemukan." << endl;
            } else {
                ElmList* Q = P->next;

                P->next = Q->next;
                dealokasi(Q);
            }
        }
    }

    // Mengubah nilai data
    void update(List &L, int nilaiLama, int nilaiBaru) {
        ElmList* P = findElm(L, nilaiLama);

        if (P == nullptr) {
            cout << "Nilai tidak ditemukan." << endl;
        } else {
            P->info = nilaiBaru;
        }
    }

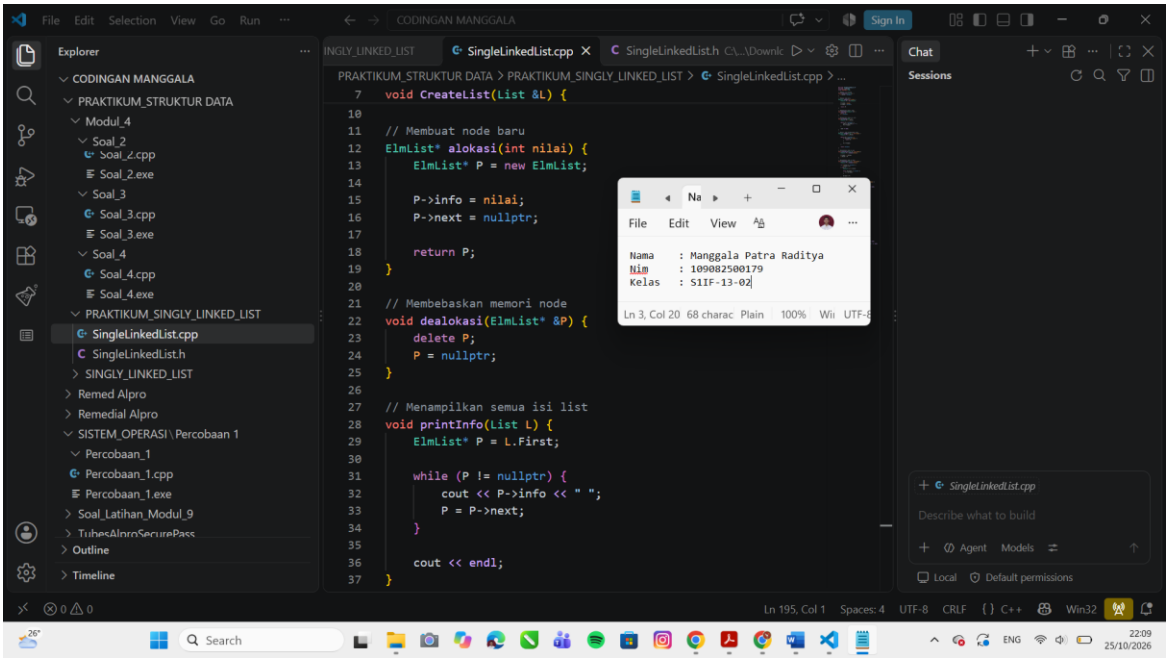
    // Menghitung jumlah node
    int nbList(List L) {
        int jumlah = 0;
        ElmList* P = L.First;

        while (P != nullptr) {
            jumlah++;
            P = P->next;
        }

        return jumlah;
    }

    // Menghapus semua node
    void delAll(List &L) {
        while (L.First != nullptr) {
            delFirst(L);
        }
    }
```

Screenshoot program



Deskripsi program

File SingleLinkedList.cpp merupakan file implementasi dari struktur data Singly Linked List yang berisi fungsi-fungsi untuk mengelola elemen dalam list. Fungsi CreateList digunakan untuk membuat list kosong, alokasi untuk membuat node baru dan menyimpan nilai, serta dealokasi untuk membebaskan memori node. Fungsi printInfo menampilkan seluruh nilai dalam list, sedangkan findElm mencari node berdasarkan nilai tertentu. Fungsi insertFirst, insertLast, dan insertAfter digunakan untuk menambahkan node di awal, akhir, atau setelah node tertentu. Sementara itu, delFirst, delLast, delAfter, dan delP berfungsi menghapus node berdasarkan posisinya atau nilai tertentu. Fungsi update digunakan untuk mengubah nilai pada node, nbList menghitung jumlah node dalam list, dan delAll menghapus seluruh node hingga list kosong. Seluruh fungsi tersebut memanfaatkan pointer untuk menghubungkan, menelusuri, menambahkan, dan menghapus node serta mengelola memori secara dinamis.

Source Code (main.cpp)

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* hasil;
    ElmList* P;
    int total = 0;

    CreateList(L);
```

```
// 1. Insert First
insertFirst(L, 2);
insertFirst(L, 0);
insertFirst(L, 8);
insertFirst(L, 12);
insertFirst(L, 9);

cout << "List awal: ";
printInfo(L);

// 2. Searching
hasil = findElm(L, 8);

if (hasil != nullptr) {
    cout << "Angka 8 ditemukan." << endl;
} else {
    cout << "Angka 8 tidak ditemukan." << endl;
}

// 3. Menghitung total nilai
P = L.First;

while (P != nullptr) {
    total = total + P->info;
    P = P->next;
}

cout << "Total nilai: " << total << endl;

// 4. Insert Last
insertLast(L, 20);
cout << "Setelah insertLast(20): ";
printInfo(L);

// 5. Insert After
insertAfter(L, 12, 99);
cout << "Setelah insertAfter(12, 99): ";
printInfo(L);

// 6. Update
update(L, 99, 100);
cout << "Setelah update(99, 100): ";
printInfo(L);

// 7. Delete After
delAfter(L, 12);
cout << "Setelah delAfter(12): ";
```

```
        printInfo(L);

        // 8. Delete berdasarkan nilai
        delP(L, 8);
        cout << "Setelah delP(8): ";
        printInfo(L);

        // 9. Delete First
        delFirst(L);
        cout << "Setelah delFirst: ";
        printInfo(L);

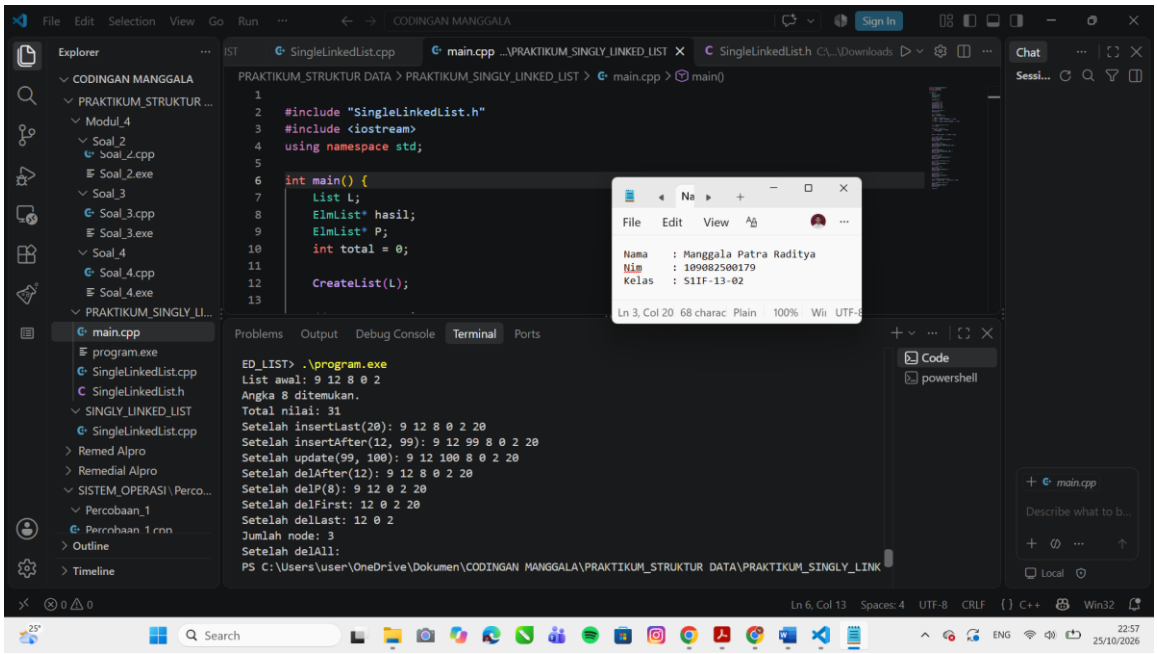
        // 10. Delete Last
        delLast(L);
        cout << "Setelah delLast: ";
        printInfo(L);

        // 11. Menghitung jumlah node
        cout << "Jumlah node: " << nbList(L) << endl;

        // 12. Menghapus semua node
        delAll(L);
        cout << "Setelah delAll: ";
        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

File main.cpp merupakan program utama yang digunakan untuk menguji berbagai operasi pada struktur data Singly Linked List. Program diawali dengan mendeklarasikan variabel List L, pointer hasil dan P, serta variabel total untuk menyimpan jumlah nilai seluruh elemen. Fungsi CreateList(L) digunakan untuk membuat list kosong, kemudian fungsi insertFirst memasukkan data 2, 0, 8, 12, dan 9 ke bagian awal list sehingga menghasilkan urutan 9, 12, 8, 0, 2. Fungsi printInfo menampilkan isi list, sedangkan findElm(L, 8) digunakan untuk mencari elemen bernilai 8. Selanjutnya, program menghitung total nilai seluruh elemen menggunakan perulangan while dan menghasilkan nilai 31. Program kemudian menguji fungsi insertLast untuk menambahkan angka 20 di akhir list, insertAfter untuk menyisipkan angka 99 setelah angka 12, serta update untuk mengubah angka 99 menjadi 100. Setelah itu, fungsi delAfter menghapus elemen setelah angka 12, delP menghapus elemen bernilai 8, delFirst menghapus elemen pertama, dan delLast menghapus elemen terakhir. Fungsi nbList digunakan untuk menghitung jumlah node yang tersisa, sedangkan delAll menghapus seluruh node dalam list. Dengan demikian, program ini menunjukkan penerapan operasi dasar Singly Linked List, mulai dari pembuatan list, penambahan, pencarian, pengubahan, penghapusan, hingga penghitungan jumlah elemen.

2. Soal 2

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* hasil;

    CreateList(L);

    insertFirst(L, 2);
    insertFirst(L, 0);
    insertFirst(L, 8);
    insertFirst(L, 12);
    insertFirst(L, 9);

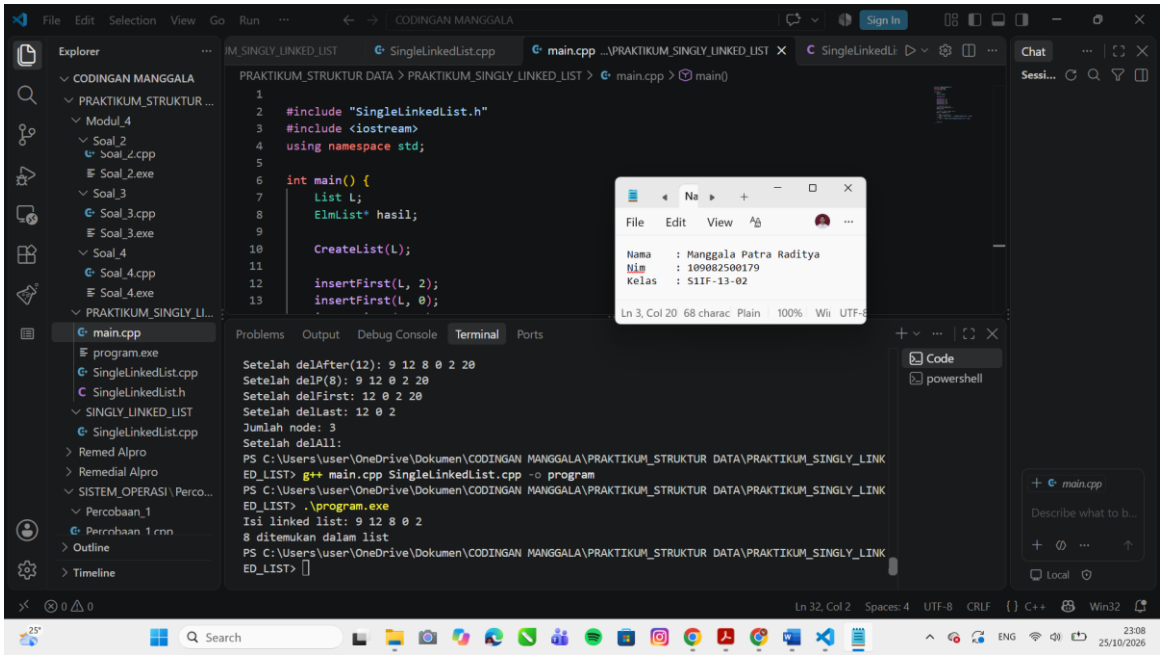
    // Menampilkan isi list
    cout << "Isi linked list: ";
    printInfo(L);

    // Mencari elemen dengan info 8
    hasil = findElm(L, 8);
```

```
    if (hasil != nullptr) {
        cout << hasil->info << " ditemukan dalam list" <<
endl;
    } else {
        cout << "Elemen tidak ditemukan dalam list" << endl;
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Program main.cpp digunakan untuk menguji operasi Singly Linked List, dimulai dengan mendeklarasikan list L dan pointer hasil, kemudian fungsi CreateList(L) digunakan untuk membuat list kosong. Selanjutnya, fungsi insertFirst memasukkan nilai 2, 0, 8, 12, dan 9 ke bagian awal list sehingga menghasilkan urutan 9, 12, 8, 0, 2. Fungsi printInfo(L) digunakan untuk menampilkan seluruh elemen dalam list. Setelah itu, fungsi findElm(L, 8) mencari elemen yang memiliki nilai 8 dan menyimpan alamat node yang ditemukan ke dalam pointer hasil. Percabangan if-else memeriksa apakah elemen berhasil ditemukan. Jika hasil tidak bernilai nullptr, program menampilkan pesan bahwa angka 8 ditemukan dalam list. Sebaliknya, jika elemen tidak ditemukan, program menampilkan pesan bahwa elemen tidak ditemukan. Fungsi return 0 menandakan bahwa program selesai dijalankan dengan baik.

3. Soal 3

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* P;
    int total = 0;

    // Membuat list kosong
    CreateList(L);

    // Memasukkan elemen ke dalam list
    insertFirst(L, 2);
    insertFirst(L, 0);
    insertFirst(L, 8);
    insertFirst(L, 12);
    insertFirst(L, 9);

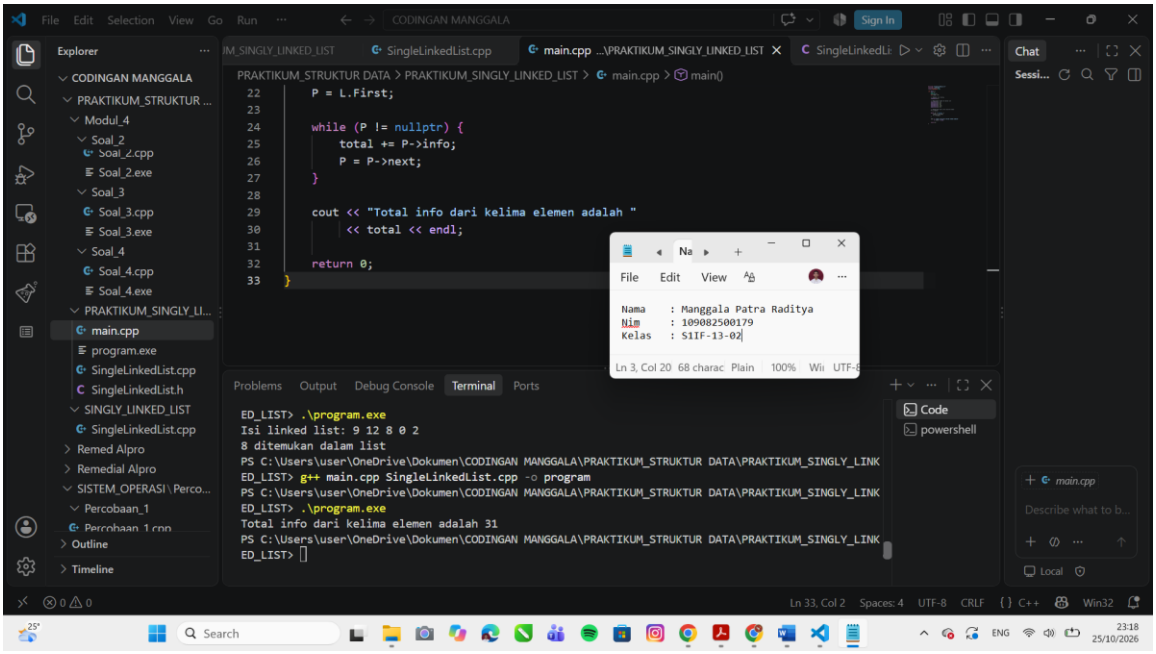
    // Menghitung total info seluruh elemen
    P = L.First;

    while (P != nullptr) {
        total += P->info;
        P = P->next;
    }

    cout << "Total info dari kelima elemen adalah "
         << total << endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Program main.cpp digunakan untuk menghitung jumlah total nilai seluruh elemen dalam Singly Linked List. Program diawali dengan membuat list kosong menggunakan fungsi CreateList(L), kemudian memasukkan nilai 2, 0, 8, 12, dan 9 menggunakan fungsi insertFirst, sehingga urutan elemen menjadi 9, 12, 8, 0, 2. Pointer P diarahkan ke node pertama melalui L.First, lalu perulangan while digunakan untuk menelusuri setiap node selama P tidak bernilai nullptr. Pada setiap perulangan, nilai P->info ditambahkan ke variabel total, kemudian pointer P berpindah ke node berikutnya. Setelah semua elemen dijumlahkan, program menampilkan hasil total sebesar 31.